- Latest and stable version of Docker
- Ruby version >=2.6, along with Test-Kitchen, Kitchen-Docker and Kitchen-Salt libraries
- Python version 3.x, along with PyTest and TestInfra modules

```
[root@ip-172-31-31-154 kitchen-salt]# docker --version
Docker version 1.13.1, build 7d71120/1.13.1
[root@ip-172-31-31-154 kitchen-salt]# ruby --version
ruby 2.7.6p219 (2022-04-12 revision c9c2245c0a) [x86_64-
linux]
[root@ip-172-31-31-154 kitchen-salt]#gem --version
3.0.3.1
[root@ip-172-31-31-154 kitchen-salt]#bundle --version
Bundler version 2.3.24
[root@ip-172-31-31-154 kitchen-salt]# python3 --version
Python 3.6.8
[root@ip-172-31-31-154 kitchen-salt]# pip3.6 install -r
requirements.txt
.................................................
Successfully installed attrs-22.1.0 certifi-2022.9.24
charset-normalizer-2.0.12 idna-3.4 importlib-
metadata-4.8.3 iniconfig-1.1.1 packaging-21.3
pluggy-1.0.0 py-1.11.0 pyparsing-3.0.9 pytest-7.0.1
pytest-testinfra-6.8.0 requests-2.27.1 testinfra-6.0.0
tomli-1.2.3 typing-extensions-4.1.1 urllib3-1.26.12 zipp-
3.6.0
```

How does it work?

To design a seamless testing workflow, we will use KitchenSalt in conjunction with TestInfra to improve the quality of states and reduce errors while deploying Salt code in a production environment.

KitchenSalt is built on top of Test Kitchen, which was initially developed to test and validate Chef workflows. It is a provisioner for SaltStack, which permits users to leverage Test Kitchen to validate the SaltStack environment locally without a Salt master or minions. On the other hand, TestInfra is a Python testing module written on top of the popular Pytest testing framework. With the help of TestInfra, users can run integration or unit tests to validate the actual state of the servers configured by SaltStack.

Figure 1 illustrates the logical representation of the entire workflow, where we will spin up a Docker container using KitchenSalt and apply Salt states or formulas to it. Once the provisioning is completed, a few integration tests will be run with the help of TestInfra to validate the state of the provisioned container.

Prepare Webserver formula

For this demonstration, we will need a working Salt formula. So, let's create a basic one to install and configure a Webserver instance. The formula consists of the following three essential states.

- *install.sls:* To install Webserver (Apache) packages on the instance
- *configure.sls:* To set up Webserver conf and *index.html* page
- *service.sls:* To start Webserver service once setup is completed

```
[root@ip-172-31-31-154 kitchen-salt]# cat webserver/
install.sls
{% from "webserver/map.jinja" import vars with context %}
{{ sls }}.pkg:
  pkg.installed:
    - name: {{ vars['package'] }}
{{ sls }}.group:
  group.present:
    - name: {{ pillar['app_user'] }}
    - gid: {{ pillar['gid'] }}
    - require:
      - pkg: {{ sls }}.pkg
{{ sls }}.user:
  user.present:
    - name: {{ pillar['app_user'] }}
    - uid: {{ pillar['uid'] }}
    - gid: {{ pillar['gid'] }}
    - home: {{ pillar['doc_root'] }}
    - shell: /bin/nologin
    - allow_uid_change: true
```

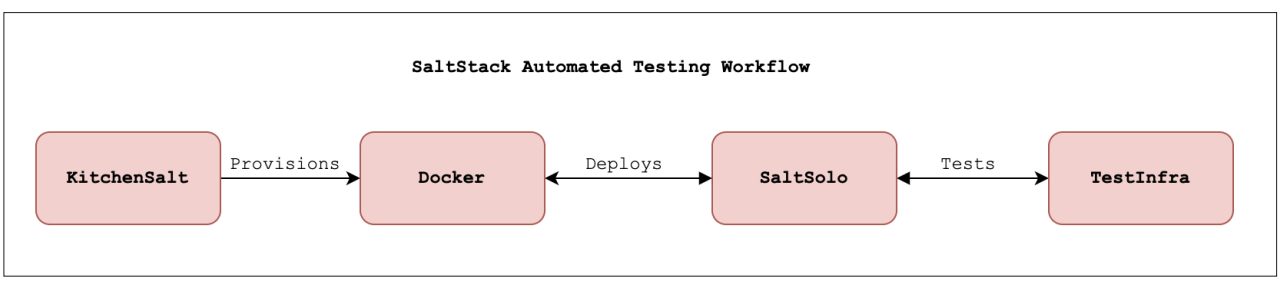

Figure 1: Logical representation of the entire workflow